

Advanced Texturing

Computer Graphics
Instructor: Sungkil Lee

Today

- **Advanced texture mapping**

- Environment mapping
- Normal/bump mapping
- Displacement mapping with tessellation

- **Shadow mapping**

- Hard shadows
- Soft shadow mapping

Advanced Texture Mapping

Implication in Blinn-Phong Illumination Model

$$I'_r = \mathbf{k}_a I_a + \max(\mathbf{k}_d (\mathbf{l} \cdot \mathbf{n}) I_d, 0) + \max(\mathbf{k}_s (\mathbf{n} \cdot \mathbf{h})^\beta I_s, 0)$$

object마다 주는 물체의 특성들
→ 상수처럼 정의되어 있어 변화하는
무언가를 표현할 때 디테일하지 못함
→ 변화시킬 수 있는 건 n 과 β 정도

- **Surface properties are defined as images.**

- We usually define $\mathbf{k}_a, \mathbf{k}_d, \mathbf{k}_s$ with manually chosen constants.
- With texture mapping, we define $\mathbf{k}_a, \mathbf{k}_d, \mathbf{k}_s$ using images.
- Recall that the image is a sampled representation of a continuous function, which can be efficient to represent varying surface properties.

- **Common types of texture mapping**

- Diffuse mapping ($\mathbf{k}_d$)
- Environment mapping (reflection mapping)
- Bump/normal mapping ($\mathbf{n}$)
- Specular Mapping (β), but rare in practice

surface properties object마다 상수로
정의했는에 surface마다 정의해서 image에 각각
 k_a, k_d, k_s 는 각각다른 값만 image에 쓰면
좀 더 복잡한 것 표현가능

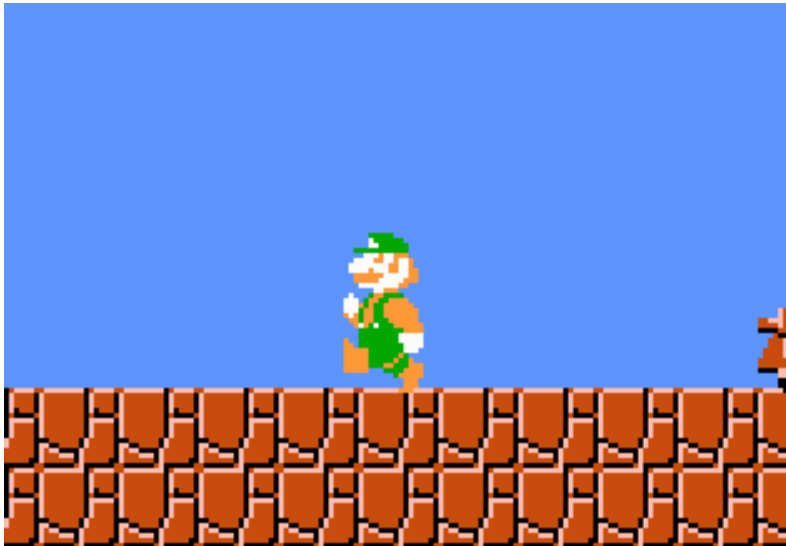
셰이딩 값이 일한 수면만 보일듯 image는
값과 빛을 sampling하는 것이라서 위치마다 변화
→ 변화되는 것이라 이름이 따로 있음

Diffuse mapping은 k_d 가 바뀔

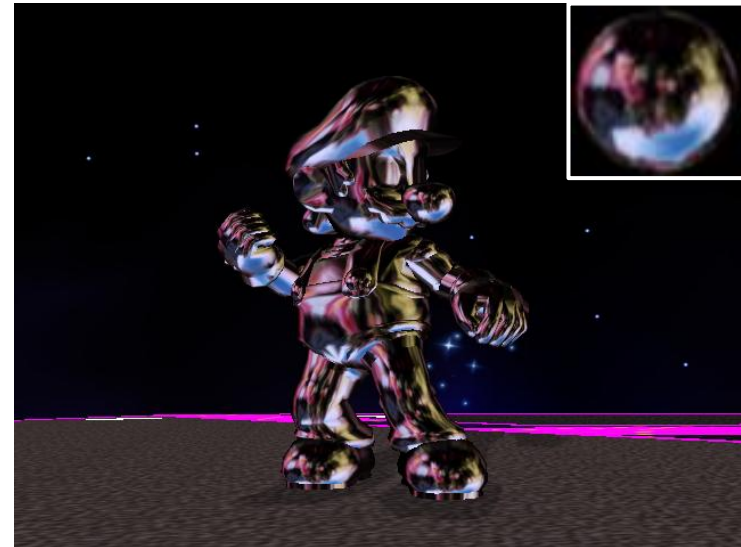
보통 texture mapping은 diffuse mapping을
이야기하러 이리 다른것의 예시도 따로 다루지 않음

Environment Mapping

- **Environment mapping (or reflection mapping)** *가장, 중요*
 - Uses a picture of the environment for highly specular reflection
 - The colors of environments are mapped to the surface according to the reflector against the view vector. *scene의 그림으로 반사 효과를 만들*
 - It can be considered as a simple approximation of ray tracing.



Shiny invincible Mario



Emulation using env. mapping

Environment Mapping

- **Omnidirectional texture mapping** 모든 방향을 다 볼
 - Environment mapping requires omnidirectional images, which can encode 360 surroundings (similar to 360 VR).
- **Common ways of omnidirectional texture mapping**
 - Sphere map 구
 - Cube map 큐브
 - (Dual)-paraboloid map 구와 큐브 중간

Sphere (Environment) Mapping

- **Environment mapping:**
 - Environment maps are modeled as a sphere or cube.



sphere map

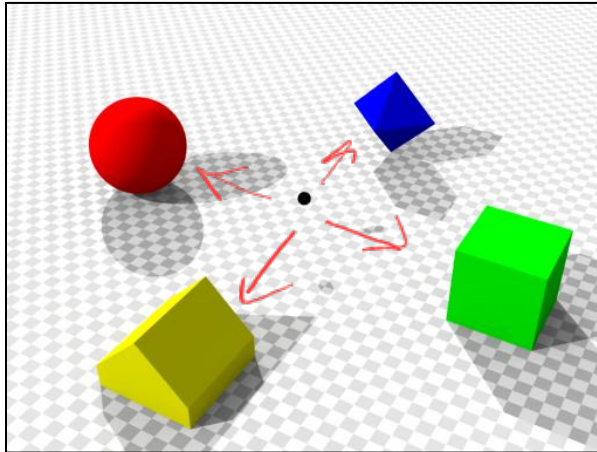


rendering examples with sphere-map reflections

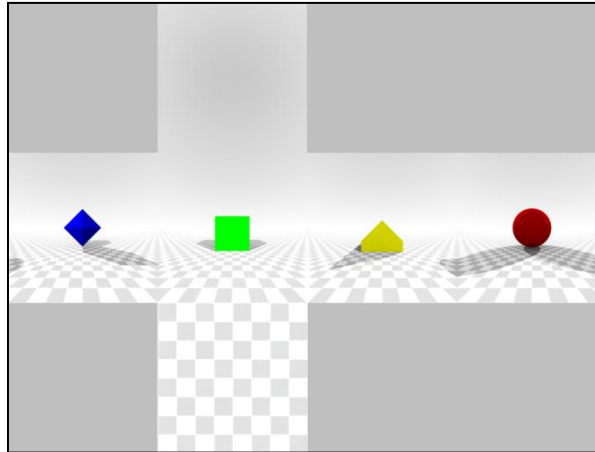
Cube (Environment) Mapping

- **Cube mapping:**

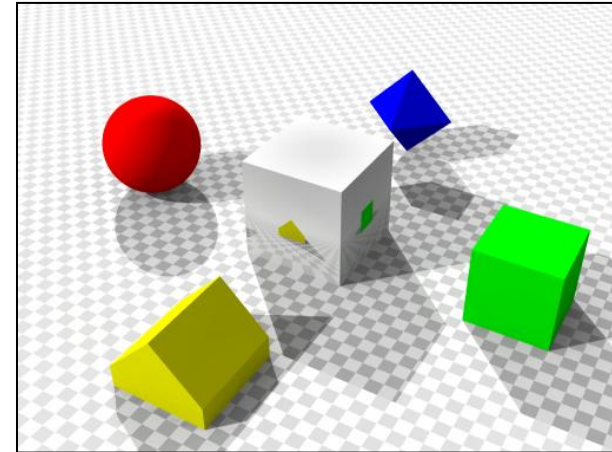
- environment mapping via a 6-face cube map *6개의 image 필요*
- Could be a static texture or dynamically-generated for every frame



place a viewer in a scene



render 6-face cube map



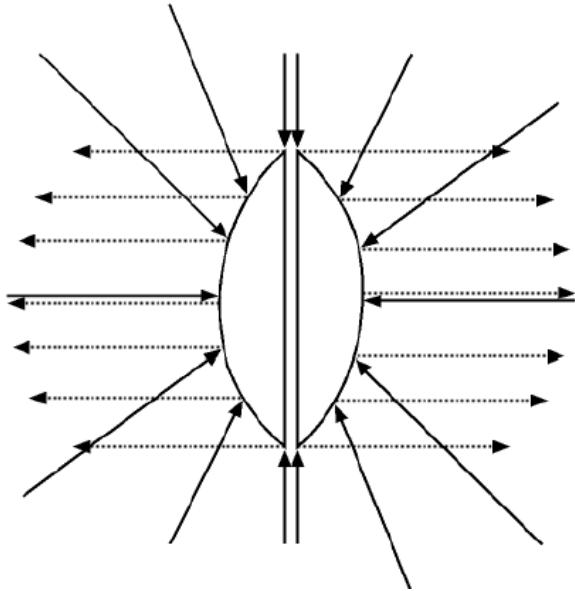
apply texture to the surface

Dual-Paraboloid (Environment) Mapping

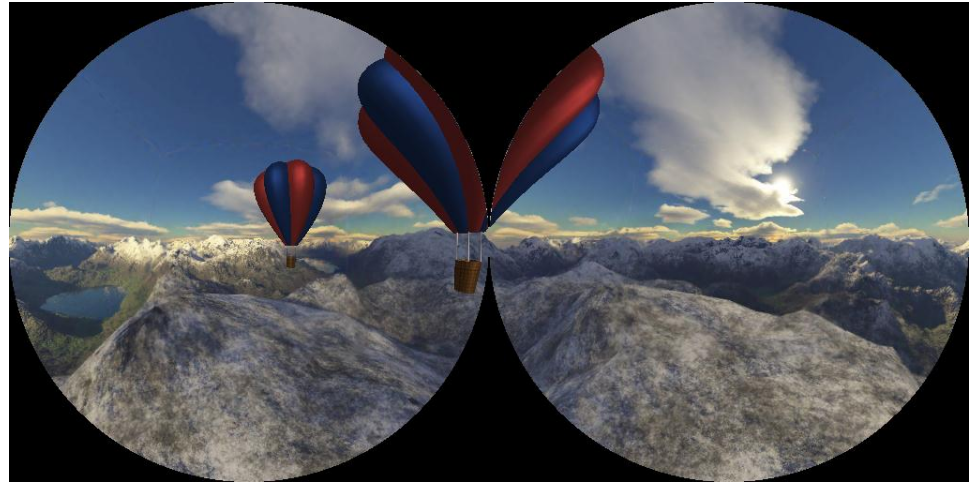
- **Paraboloid mapping** 반구를 하나의 Image로 사용

- encode a hemisphere in a single image using reflection on the paraboloid surface.
- Two images can represent the whole surrounding environment.

→ dual-paraboloid mapping



Reflection on dual-paraboloid



Example dual paraboloid maps.
(C) Imagination Tech.

Bump/Normal mapping normal vector만 바꿀

- **Bump/normal mapping** normal vector를 바꿨는데 색이 달라진
 - Emulates altering normal vectors during the rendering process
 - Efficient approximation of fine-grained geometries.



diffuse mapped



normal map



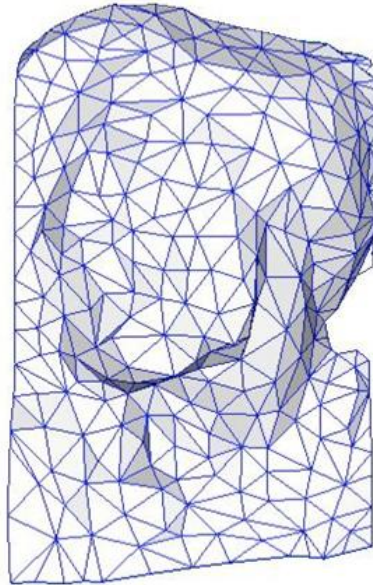
bump mapped

Examples

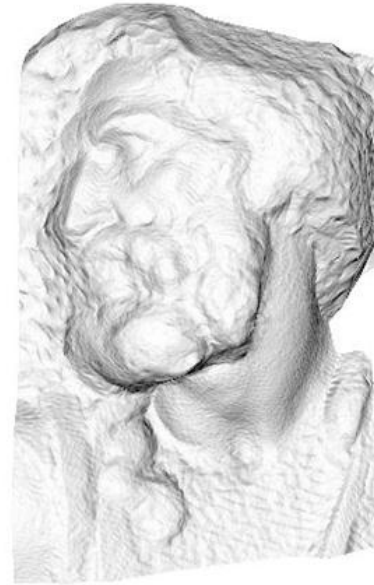
- Comparison between fine geometry and normal mapping



original mesh
4M triangles



simplified mesh
500 triangles



simplified mesh
and normal mapping
500 triangles

Another Example

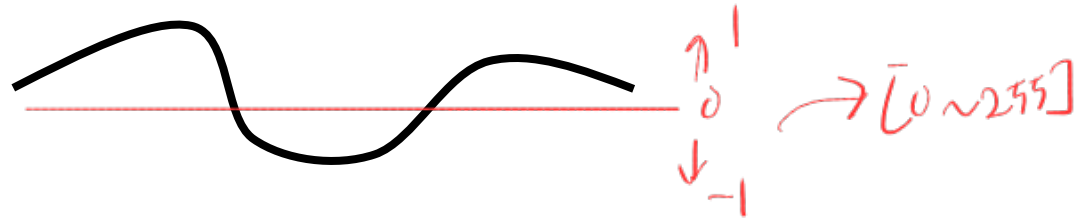


Bump Map

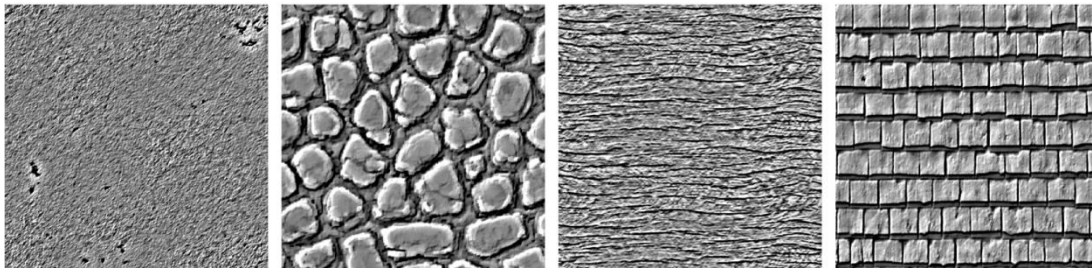
- **Bump map:**

- A 1-D height field, which records the height variation of the surface.

양면과 높고 낮아짐을 기록



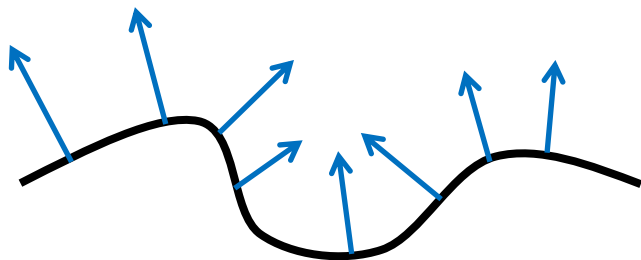
- Gray-scale image examples



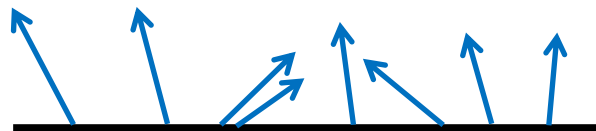
Bump/Normal mapping

- **Basic idea** *Geometry를 바꾸지 않고 높은 속임*

- Cheating your eyes without actual changes in geometric shapes.
- We change only normals and achieve as if geometry actually changes.



Normals on bumpy surface



Bumpy normals on flat surface

bump mapping은 사실 normal mapping

Normal map

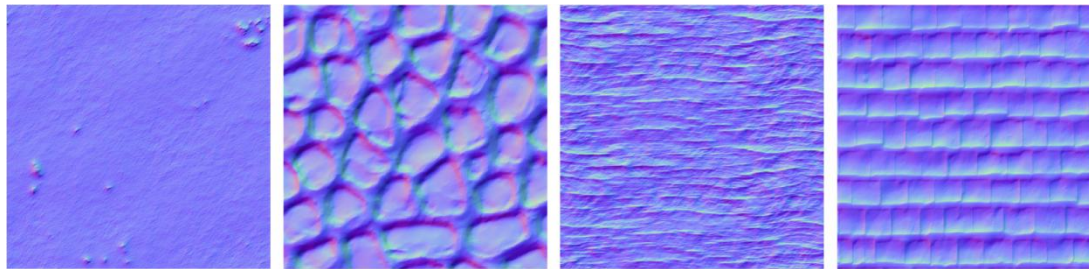
- **Normal map:**

normal vector를 인코딩하고 bump map에서 변환한 것

- encodes normal vectors in images, and is converted from a bump map.
- Actually, bump mapping refers to the normal mapping in many cases.

- **Encoding of Normals**

- A normal vector **n** can be encoded using 3-channel RGB images
- (x, y, z) normal vector is encoded in RGB; the ranges are in $[-1, 1]^3$.
 - In 8-bit images, [0, 127, 255] is mapped to [-1, 0, 1].
- So, the default normal without bumping is (0,0,1), which is encoded in (127, 127, 255) in a bump texture image.



Bump Map vs. Normal Map

- **More on bump and normal maps**

- A normal map saves computation time needed on the fly, because it provides already computed values of normal vectors from a height (= bump) map, which are needed for the shaded color determination.
- A single pixel (an RGB value) in a normal map has to be calculated from the surrounding pixels in a height map, i.e. the direction of a normal vector represented by a normal pixel is determined from the surrounding heights in a height map.

Displacement Mapping

- **Bump/normal mapping is a trick that cheats your eyes.**

- Because the normal vector is not derived from your geometry.
- A trick is revealed and the most pronounced in the edge of your geometry.

edges에서 차이가 발생



geometry



normal mapping



displacement mapping

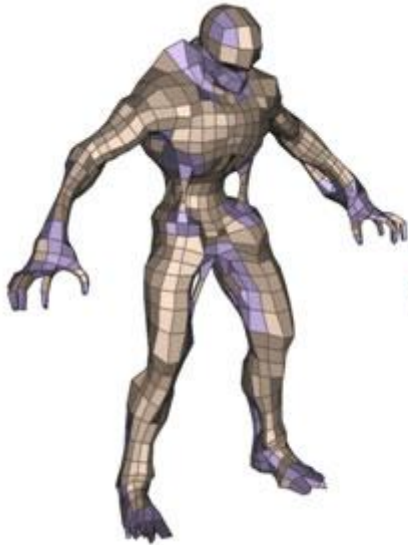
- **Displacement mapping:** height map을 사용해서 geometry를 바꾼다

- Truly relocate geometry using the height map (displacement/bump map).
- For displacement mapping, actually you need a tessellation shader, which subdivide your input meshes; this is a modern approach in many real-time applications.

Displacement Mapping + Tessellation Shaders

- **Tessellation Shaders**

- A feature of modern GPUs for on-the-fly tessellation
- Coarse control patches are tessellated (subdivided) into fine triangles, and then, high-frequency details are elevated with height map.



input control patches
가진 control patch



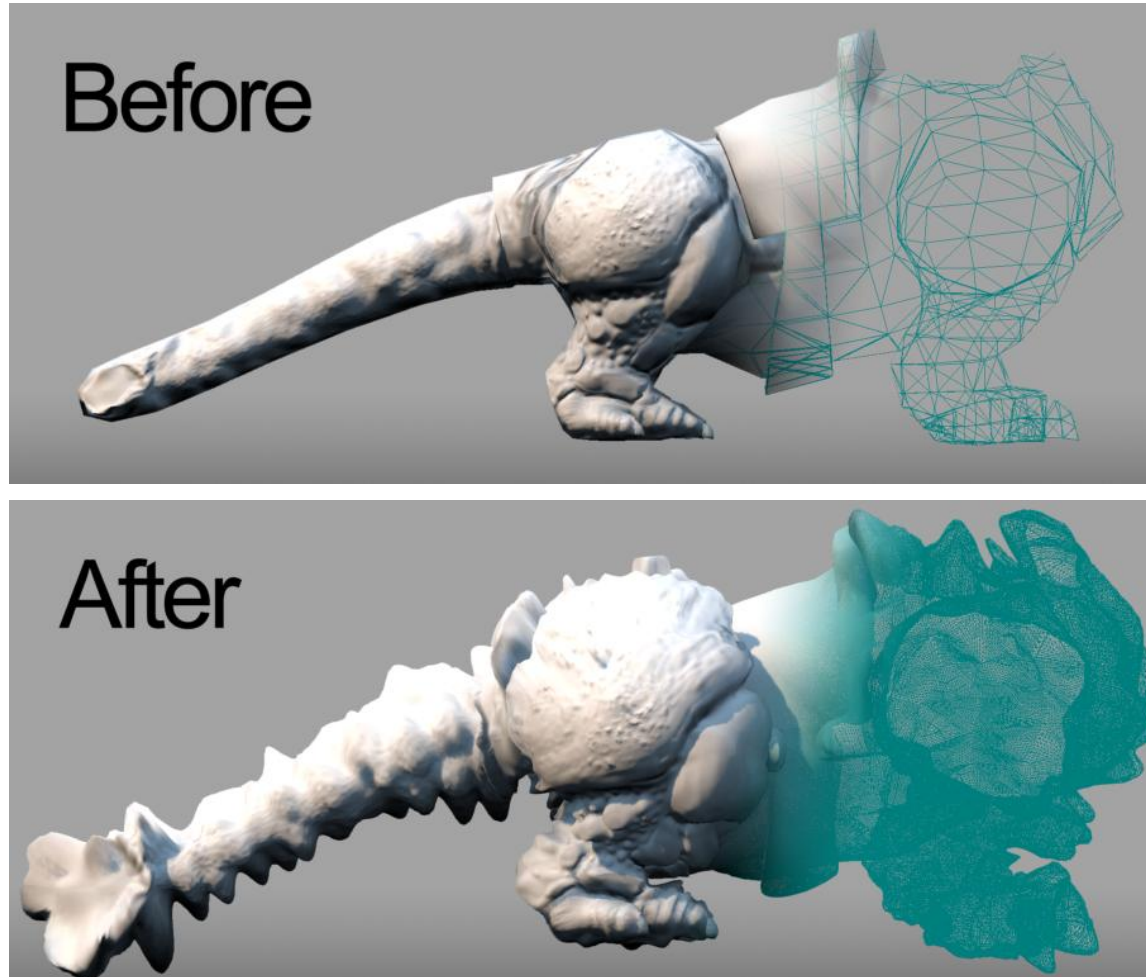
tessellation



tessellation +
displacement mapping

Displacement Mapping + Tessellation Shaders

- More examples



Shadow Mapping

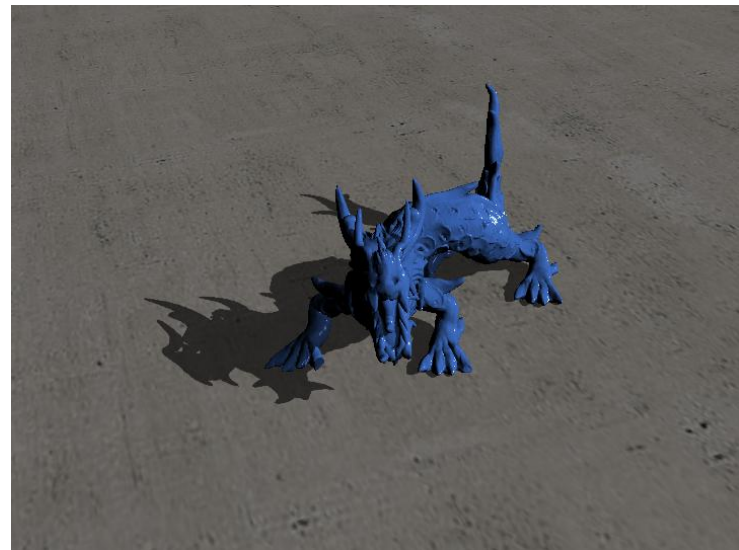
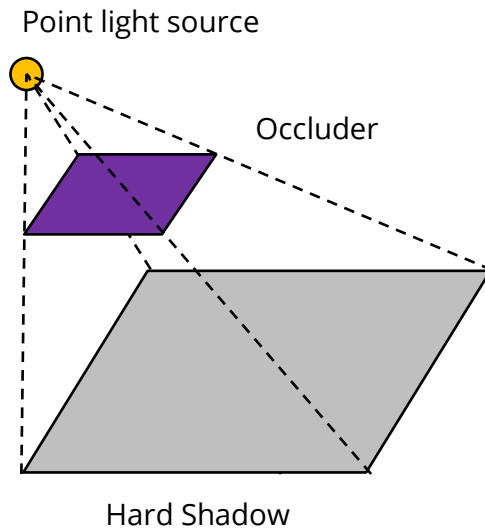
Shadows

- **Shadows**

- Dark areas where the lights are obstructed by occluders.
- Shadows create more realistic images.

- **Hard shadows**

- A point light source makes hard shadows.



Shadows

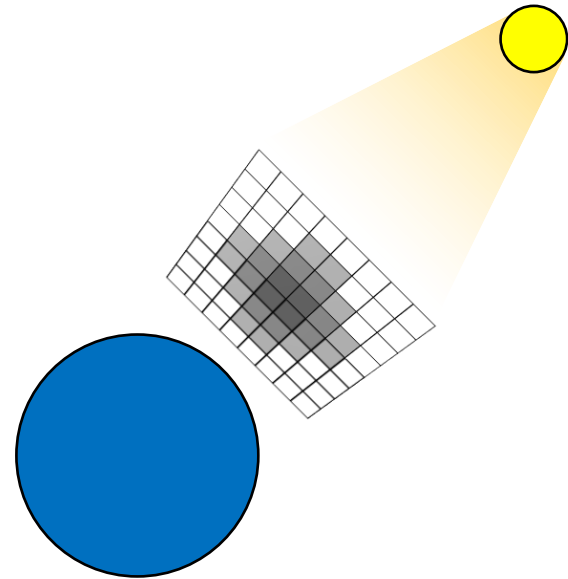
- **Two basic approaches**

- Shadow volume: geometry-based methods
- Shadow mapping: image-based methods
 - De facto standard in movie productions and 3D games

	Geometry-based	Image-based
Pros	• <u>No aliasing</u> • <u>Unlimited light field of view</u> 이때까지 영향을 미치지 않아	• Better performance • <u>Easy to implement as post-processing</u>
Cons	• Poor scene scalability <i>overhead↑</i> • Fill-rate limited	• Very hard to tweak • <u>Poor resolution scalability</u> • Shadow aliasing and acne

Algorithm

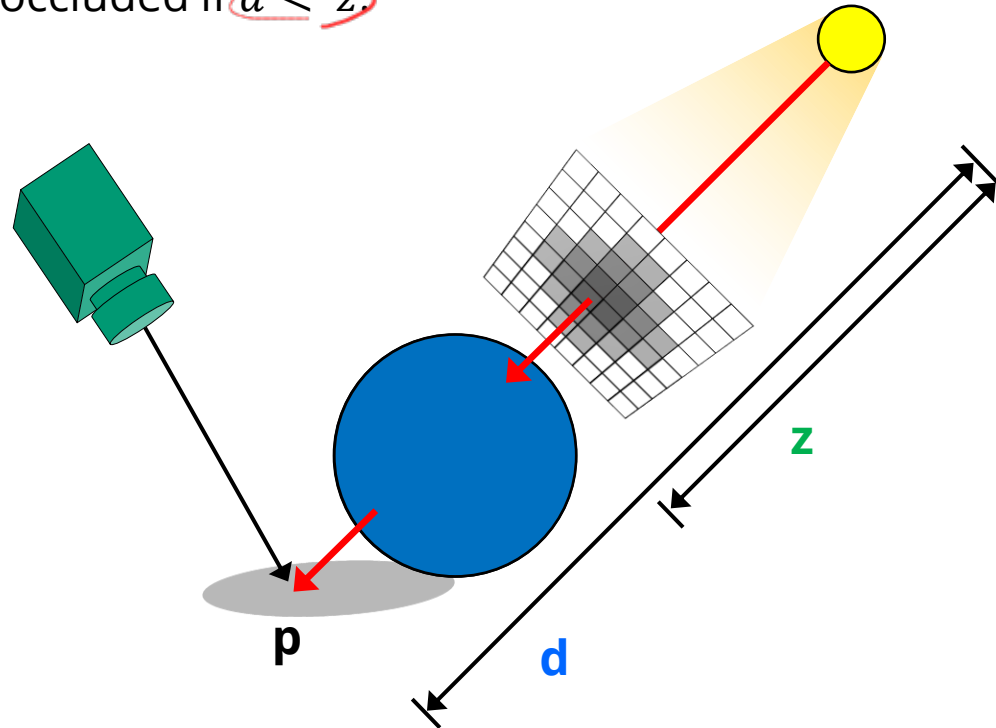
- **First pass: shadow map rendering** *광원에서 raster하는 것*
 - Render the depth image (shadow map) from the light viewpoint. *depth는 거칠*



Algorithm

- **Second pass: calculating shadow factors**

- Calculate shadow factors for each fragment that determines whether the fragment is shaded.
 - Compare depth values of the current fragment (z) and the shadow map (d).
 - Point p is lit (no shadows) if $d \geq z$.
 - Point p is occluded if $d < z$.

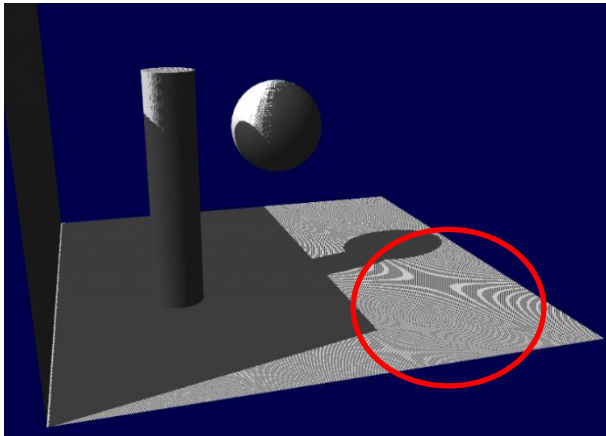


Limitations

- **Limitations of basic shadow mapping**

- Shadow acne
- Aliasing when using low-resolution shadow maps
- Unrealistic hard shadows

2F0Hk6S shadow map



Shadow acne

<http://www.opengl-tutorial.org/intermediate-tutorials/tutorial-16-shadow-mapping/>



Shadow aliasing

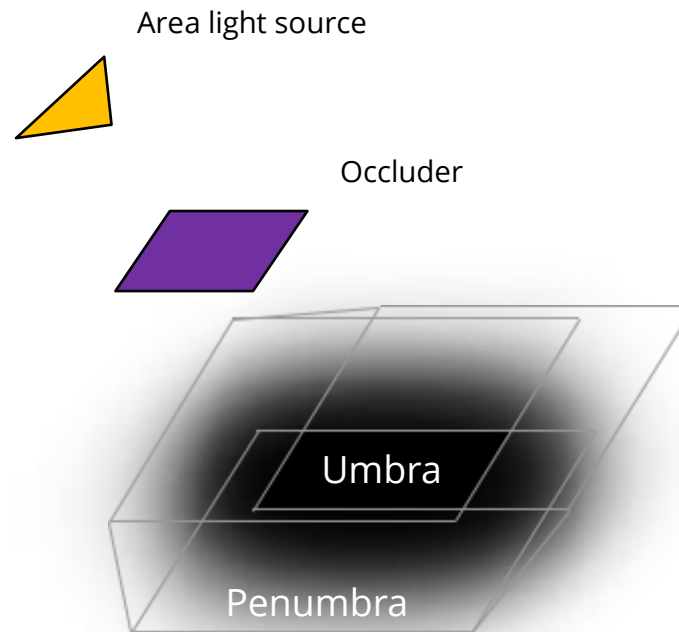
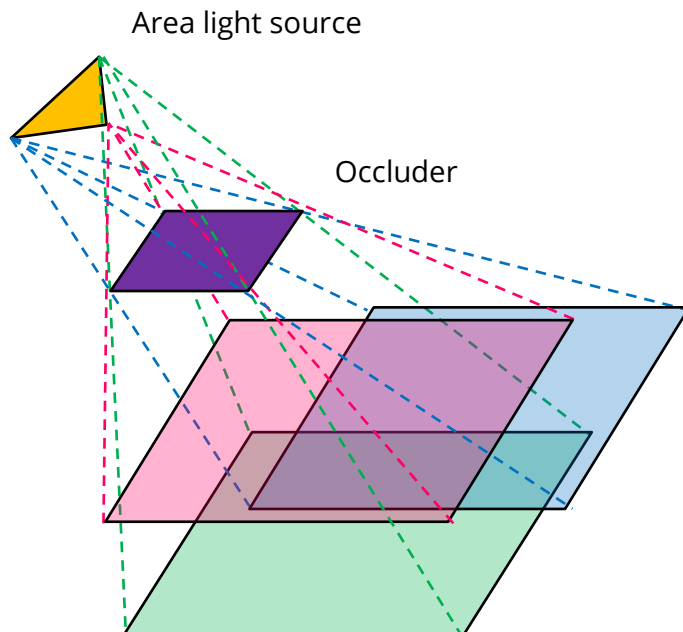
[https://msdn.microsoft.com/en-us/library/windows/desktop/ee416324\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ee416324(v=vs.85).aspx)

Soft Shadows

- **Soft shadows**

- Area light sources make soft shadows.
- Looks more realistic than hard shadows.
- Soft shadows create umbra and penumbra areas.

완전 가려짐 빛이 일부 다름

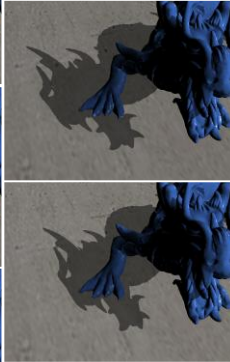
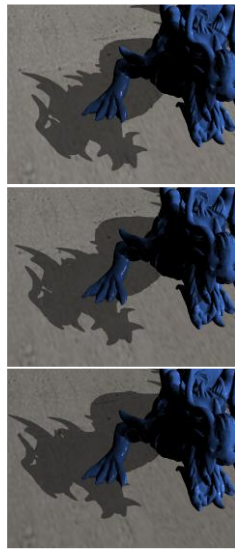
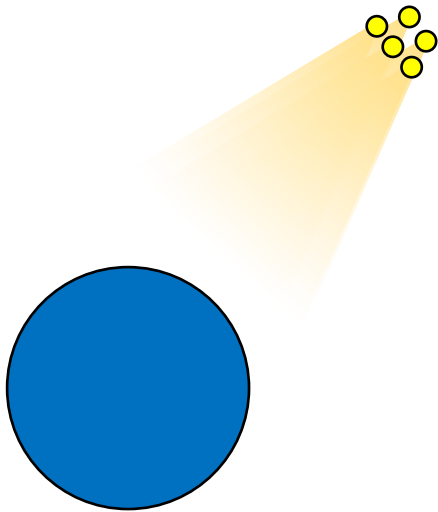


Soft Shadow mapping

- **Basic algorithm: multi-sampling of lights**

- Sample positions of point lights in the light area.
- For each point light, acquire hard shadow maps.
- Blend each shadowed results.

sampling은 point light로 만들어진
shadow factor를 합칠



Soft Shadow mapping

- **Limitations**

- Very costly with many light samples *overhead↑*
- Memory problems

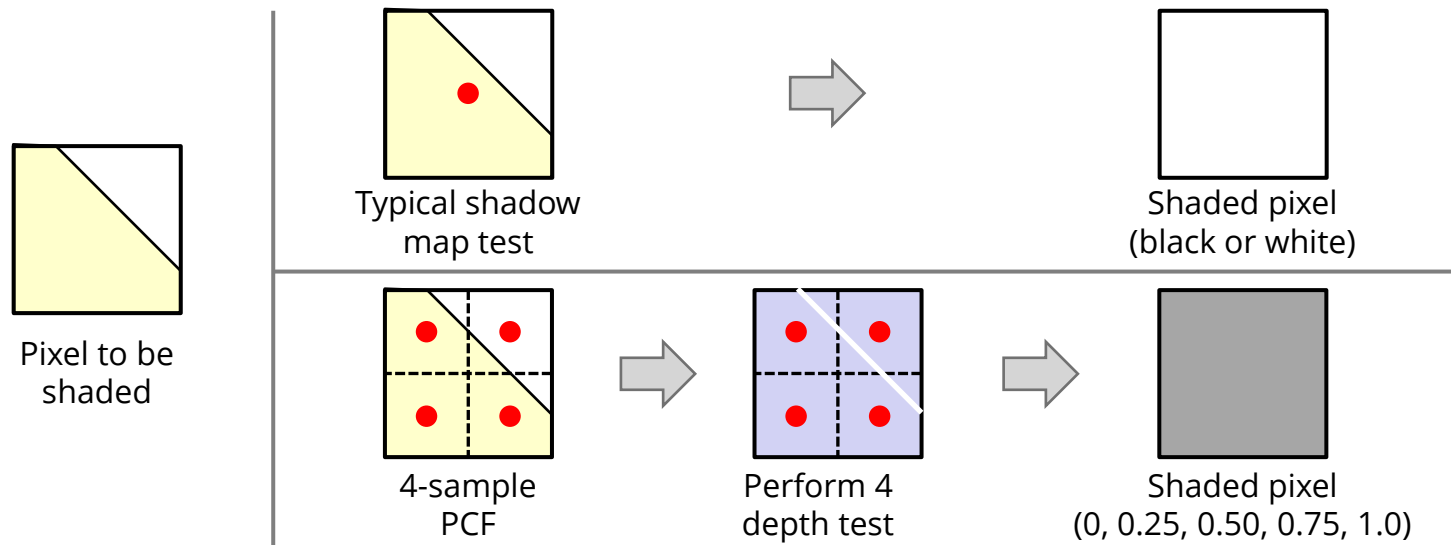
- **Approximate soft shadow mapping**

- PCSS (Percentage-closer soft shadows)
- CSSM (Convolution soft shadow maps)
- VSSM (Variance soft shadow maps)
- ESSM (Exponential soft shadow maps)

Soft Shadow mapping

- **Examples of Crytek's shadow method**

- Percentage-closer filtering (PCF) with Poisson multi-sampling.



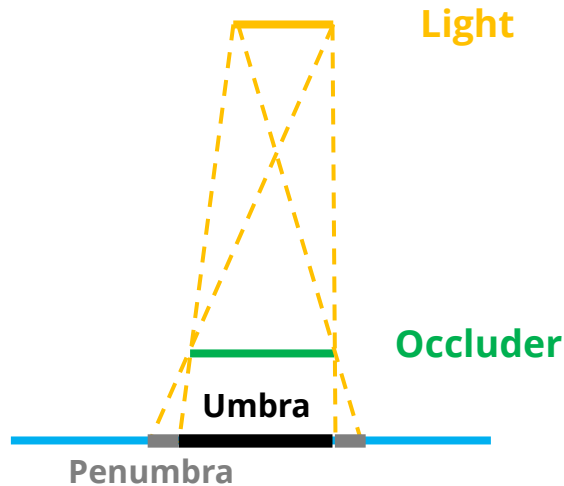
Percentage-closer filtering, Fernando, SIGGRAPH 2005 Course

원래는 가운데 하나만 객체에 여러 개를 찍어 shadow test하고 색을
광원이 아닌 shadow가 생기는 영역에 대해 geometry를 그려주는 것
(생기는 들린 방법) → 광원과 occluder 사이의 geometry를 그려

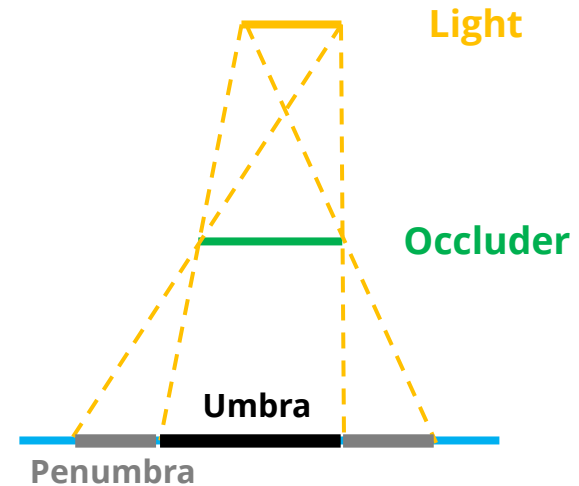
Soft Shadow mapping

- **Examples of Crytek's shadow method**

- Adjust the kernel size of PCF to approximate variable penumbra.
- Kernel size is associated with the average distance ratio to shadow casters.



Small kernel size, less softer shadow



Large kernel size, much softer shadow

Soft Shadow mapping

- **Examples of Crytek's shadow method**



Playing with Real-Time Shadows
Kasyan, SIGGRAPH 2013

Open Issues of Shadow Mapping

- **Robust**

- Stable under objects, lights and camera motion
- No light bleeding and no flickering

- **General**

- Works with all light types and dynamic geometry
- Scalable from small to large light sources

- **High quality and high performance**

- Variable penumbra and no aliasing
- Sparse sampling and good culling